Cyprus University of Technology

CSE 227 Προχωρημένη Αντικειμενοστρεφής Μεθοδολογία και Προγραμματισμός (UML, Java)

Εξαιρέσεις

Παναγιώτης Ηλία

Εξαιρέσεις (Exceptions)

Ερωτήσεις:

- Τι συμβαίνει όταν για κάποιο λάθος το πρόγραμμα παράγει runtime error;
- Πως μπορούμε να διαχειριστούμε αυτό το λάθος ώστε το πρόγραμμα να συνεχίσει την εκτέλεση του;

Απάντηση:

- Με την χρήση των **εξαιρέσεων** (**exceptions**)
  - Ορισμός: “Εξαίρεση είναι ένα συμβάν που θα μπορούσε να συμβεί κατά τη διάρκεια εκτέλεσης ενός προγράμματος και διαταράσσει την κανονική ροή των εντολών του προγράμματος”
  - “Exception” είναι συντομογραφία της φράσης "exceptional event“

Εξαιρέσεις

- Στα προγράμματα μας θα πρέπει να μπορούμε να **χειριστούμε** περιπτώσεις που το πρόγραμμα **δεν** εξελίσσεται όπως το είχαμε προβλέψει
  - Π.χ., κάνουμε μια διαίρεση και ο παρονομαστής είναι μηδέν
  - Θέλουμε να διαβάσουμε ένα ακέραιο, αλλά η είσοδος είναι ένα String
  - Θέλουμε να διαβάσουμε από ένα αρχείο αλλά δώσαμε λάθος το όνομα

- **Exceptions**
  - Οι εξαιρέσεις μας επιτρέπουν να **εντοπίσουμε το πρόβλημα** σε ένα σημείο (**throw an Exception**) και να το **χειριστούμε** σε κάποιο άλλο σημείο (**handle the Exception**)
  - Οι εξαιρέσεις είναι ένα αρκετά προχωρημένο προγραμματιστικό εργαλείο
  - Ακόμη κι αν δεν τις χρησιμοποιήσετε, εμφανίζονται σε διάφορες βιβλιοθήκες της Java, οπότε θα πρέπει να ξέρετε να τις χειρίζεστε

Σφάλματα προγραμμάτων

- Τα σφάλματα ενός προγράμματος προκαλούνται από διάφορες καταστάσεις, όπως:
  - Αποτυχία δημιουργίας μνήμης (new)
  - array[-1] (Πρόσβαση σε μη επιτρεπτή θέση μνήμης)
  - 10/0 (Μη επιτρεπτές πράξεις με αριθμούς)

- **Κάποια σφάλματα τα προλαμβάνει ο μεταγλωττιστής**
- **Κάποια άλλα πάλι ΟΧΙ**
  - Οι περισσότερες γλώσσες παρέχουν μηχανισμούς αντιμετώπισης σφαλμάτων, οι οποίοι βασίζονται περισσότερο σε προγραμματιστικές συμβάσεις και δεν εντάσσονται στη σημασιολογία της γλώσσας

Σφάλματα προγραμματιστών

- Πολύ συχνά οι προγραμματιστές δεν ακολουθούν κάποια σύμβαση αναφορικά με τα σφάλματα. Έτσι, ο κώδικάς τους:
  - Δεν ανιχνεύει σφάλματα
  - Δεν επιστρέφει τις κατάλληλες τιμές ή δεν εγείρει τις κατάλληλες σημαίες όταν ανιχνεύσει κάποιο σφάλμα
  - Δεν ελέγχει τις τιμές επιστροφής διαδικασιών ή τυχούσες σημαίες σφαλμάτων
- Η εισαγωγή κώδικα διαχείρισης σφαλμάτων στα προγράμματα, έχει ως αποτέλεσμα ο κώδικας να γίνεται πολύπλοκος και δυσανάγνωστος

Η Java εντάσσει τη διαχείριση σφαλμάτων στη σημασιολογία της γλώσσας, επιβάλλοντας την ρητή αντιμετώπισή τους

Προσέγγιση της JAVA

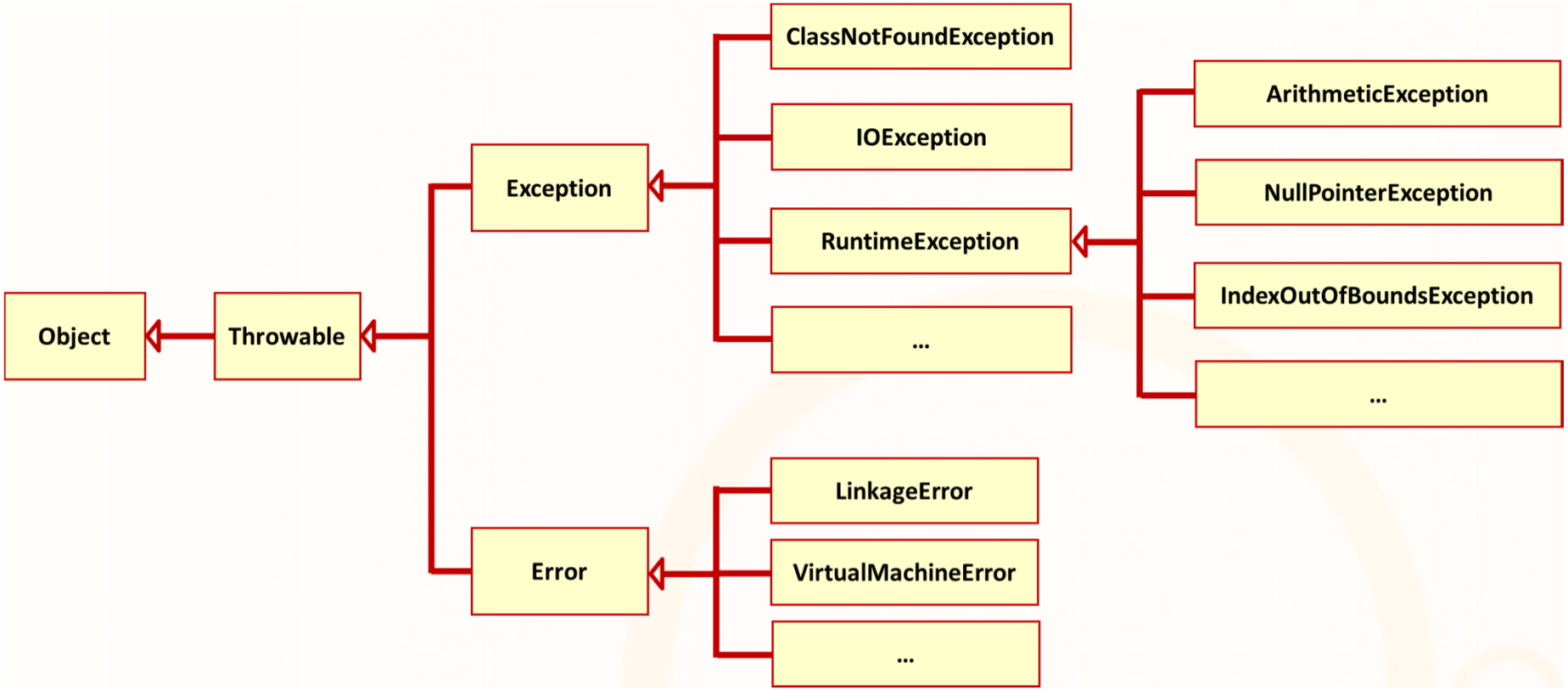
- java.lang.Object
  - java.lang.Throwable
    - java.lang.Exception



- Κλάσεις που κληρονομούν άμεσα από την **java.lang.Exception**

- Στην Java όπως και στην C# υπάρχουν κλάσεις που κληρονομούν από τους κληρονόμους της Exception
- Η τεχνική αυτή προσδίδει μία επιπλέον ιδιότητα στη διαχείριση των εξαιρέσεων
  - Αν ένα block κώδικα προστατεύεται για τη διαχείριση της **εξαίρεσης Χ** τότε προστατεύεται αυτόματα και από τους **απόγονους της Χ**

Ιεραρχία Εξαιρέσεων

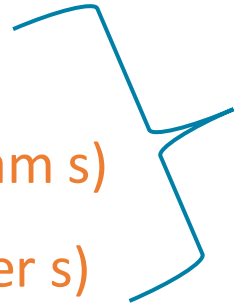


Βασικές εξαιρέσεις στη JAVA

- Η κλάση Throwable περιγράφει οτιδήποτε μπορεί να εγερθεί σαν εξαίρεση, και έχει **δύο γενικές υποκλάσεις**:
  - **Error**: αντιστοιχεί σε σφάλματα στη διάρκεια της μεταγλώττισης ή σφάλματα συστήματος, για τα οποία ο προγραμματιστής δεν χρειάζεται να τα πιάσει
  - **Exception**: ο βασικός τύπος σφάλματος ο οποίος μπορεί να εγερθεί από οποιαδήποτε μέθοδο των καθιερωμένων βιβλιοθηκών της Java
- Ο καλύτερος τρόπος για να δείτε ποιες είναι οι διάφορες εξαιρέσεις που εγείρει η Java, είναι να προηγηθείτε στο εγχειρίδιό της
- Η διαφορά των εξαιρέσεων μεταξύ τους έγκειται κατά κύριο λόγο στα διαφορετικά ονόματα που έχουν - μέσω των οποίων σηματοδοτείται το πρόβλημα στο οποίο αντιστοιχούν
- Οι διάφορες εξαιρέσεις ορίζονται στη βιβλιοθήκη **java.lang**, αλλά και σε άλλες υποστηρικτικές βιβλιοθήκες όπως util ,net, και io.

Κλάση Throwable

- **Συχνή Χρήση**
  - String toString(): Επιστρέφει την **string** περιγραφή του **throwable**
  - String getMessage(): Επιστρέφει το **μήνυμα λάθους** του throwable
  - StackTraceElement[] getStackTrace(): Δίνει προγραμματιστική πρόσβαση στις πληροφορίες του stackTrace που τυπώνει η μέθοδος printStackTrace()
  - void printStackTrace()
  - void printStackTrace(PrintStream s)
  - void printStackTrace(PrintWriter s)



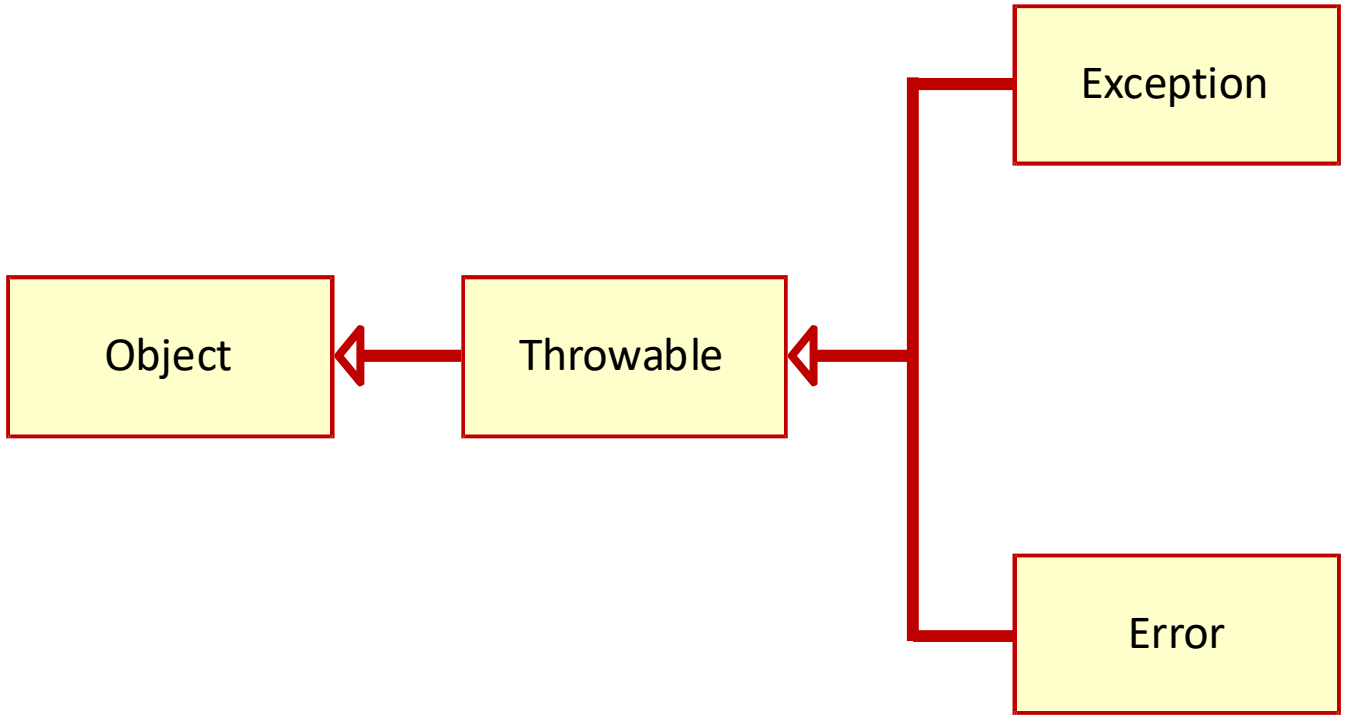
Τυπώνει το **throwable** και το **backtrace** του

Κλάση Throwable

- **Σπάνια Χρήση**
  - Throwable fillInStackTrace(): Γεμίζει το **stackTrace** εκτέλεσης
  - Throwable getCause(): Επιστρέφει την αιτία (cause) του **throwable** ή null αν η αιτία είναι unknown
  - String getLocalizedMessage(): Δημιουργεί localized περιγραφή του throwable
  - Throwable initCause(Throwable cause): Αρχικοποίησή της αιτίας (cause) του throwable με συγκεκριμένη τιμή
  - void setStackTrace(StackTraceElement[] stackTrace): Κάνει set τα αντικείμενα stack trace τα οποία επιστρέφονται από την **getStackTrace()** και τυπώνονται από την **printStackTrace()**

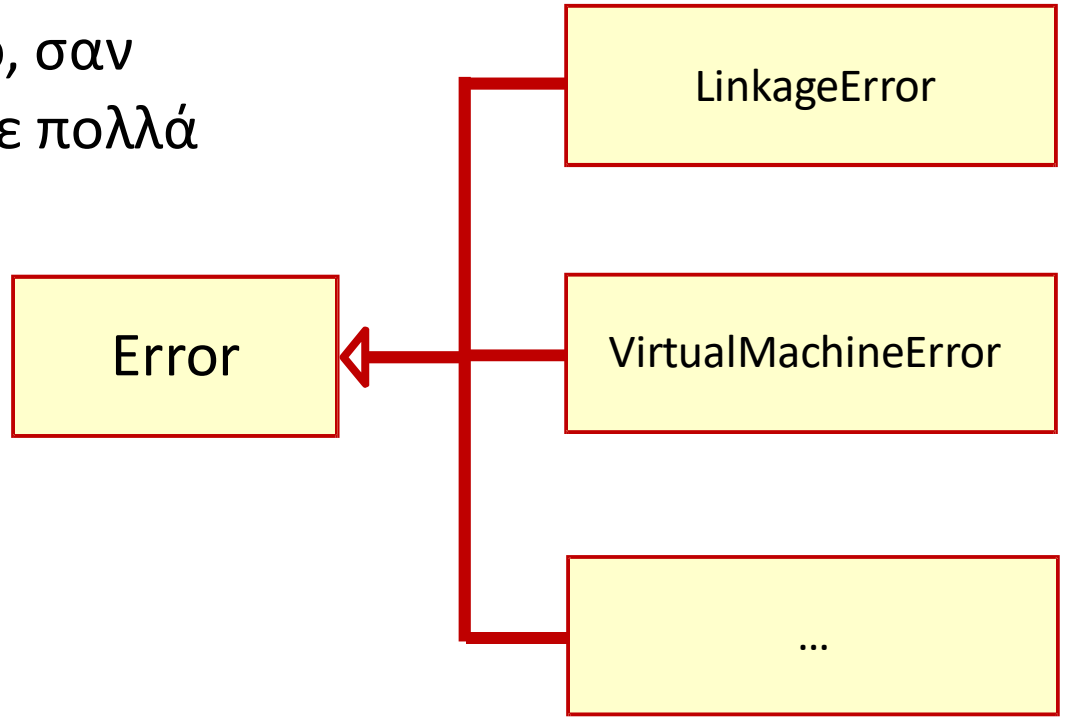
Είδη Εξαιρέσεων (Exception Types)

- Δύο βασικά Είδη Exceptions
  - Exception
  - Error



Λάθη Συστήματος (System Errors)

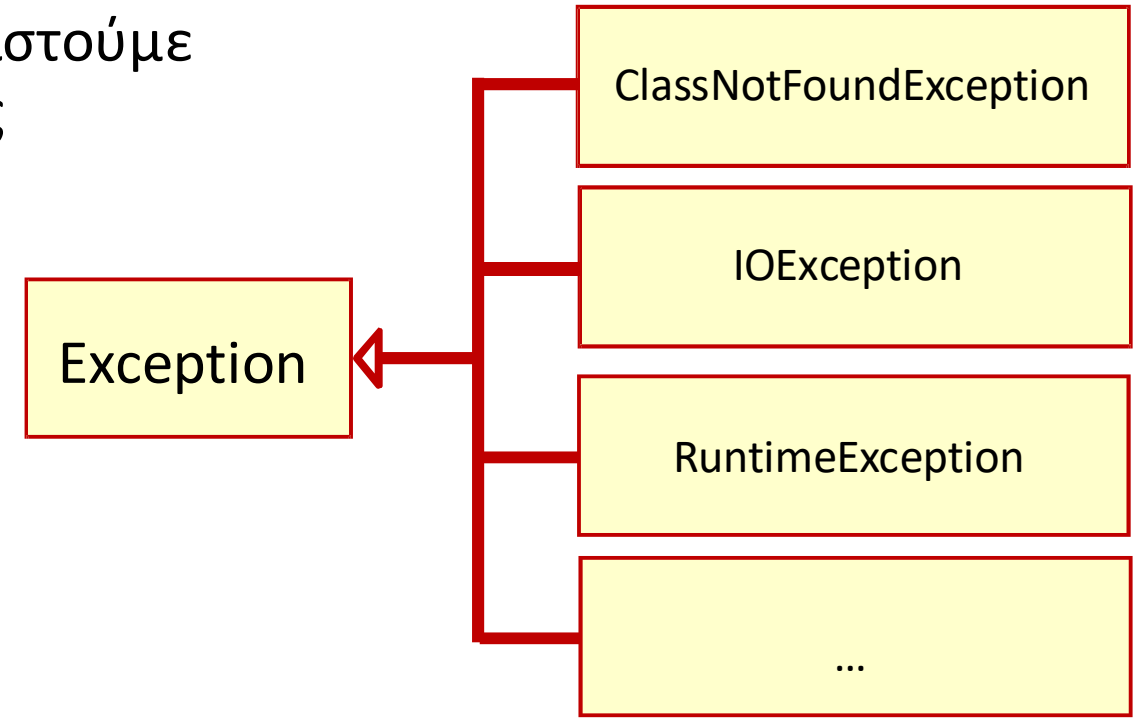
- Λάθη Συστήματος (system errors): ρίχνονται από το JVM και αναπαρίστανται από την κλάση Error
- Είναι εσωτερικά λάθη του συστήματος
- Συμβαίνουν σπάνια, αλλά αν συμβεί κάποιο, σαν προγραμματιστές δεν μπορούμε να κάνουμε πολλά



Εξαιρέσεις (Exceptions)

- **Exceptions**: αναπαριστούν λάθη τα οποία πηγάζουν από το πρόγραμμα (**Runtime Exceptions**) και εξωτερικές οντότητες

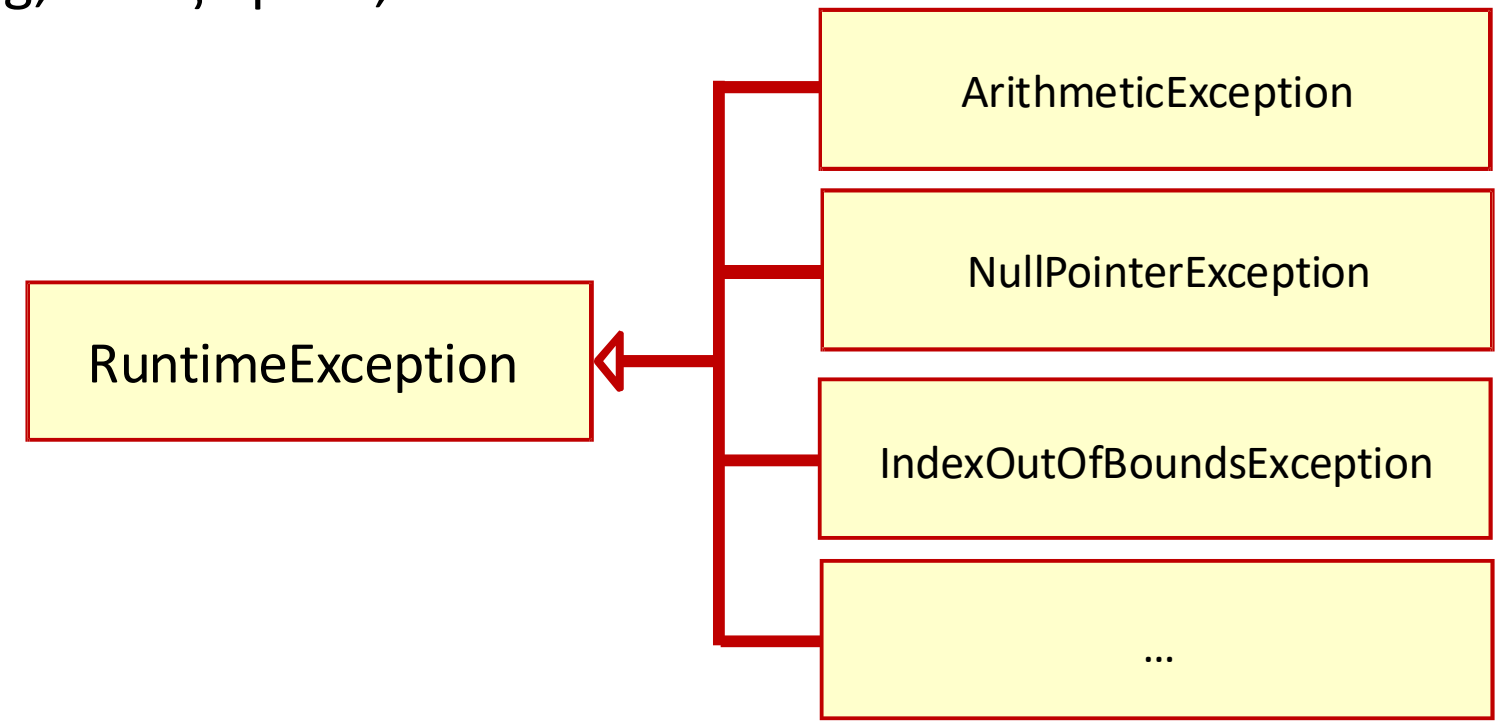
- Τα λάθη αυτά μπορούμε να τα διαχειριστούμε μέσα από το πρόγραμμα με διάφορους τρόπους όπως **try{} - catch{}**



Εξαιρέσεις Πραγματικού Χρόνου (Runtime Exceptions)

- **RuntimeExceptions**: αναπαριστούν λάθη τα οποία πηγάζουν από το πρόγραμμα

- Παραδείγματα: λάθος casting, εκτός ορίων, αριθμητικά λάθη, κτλ.



Runtime Exceptions

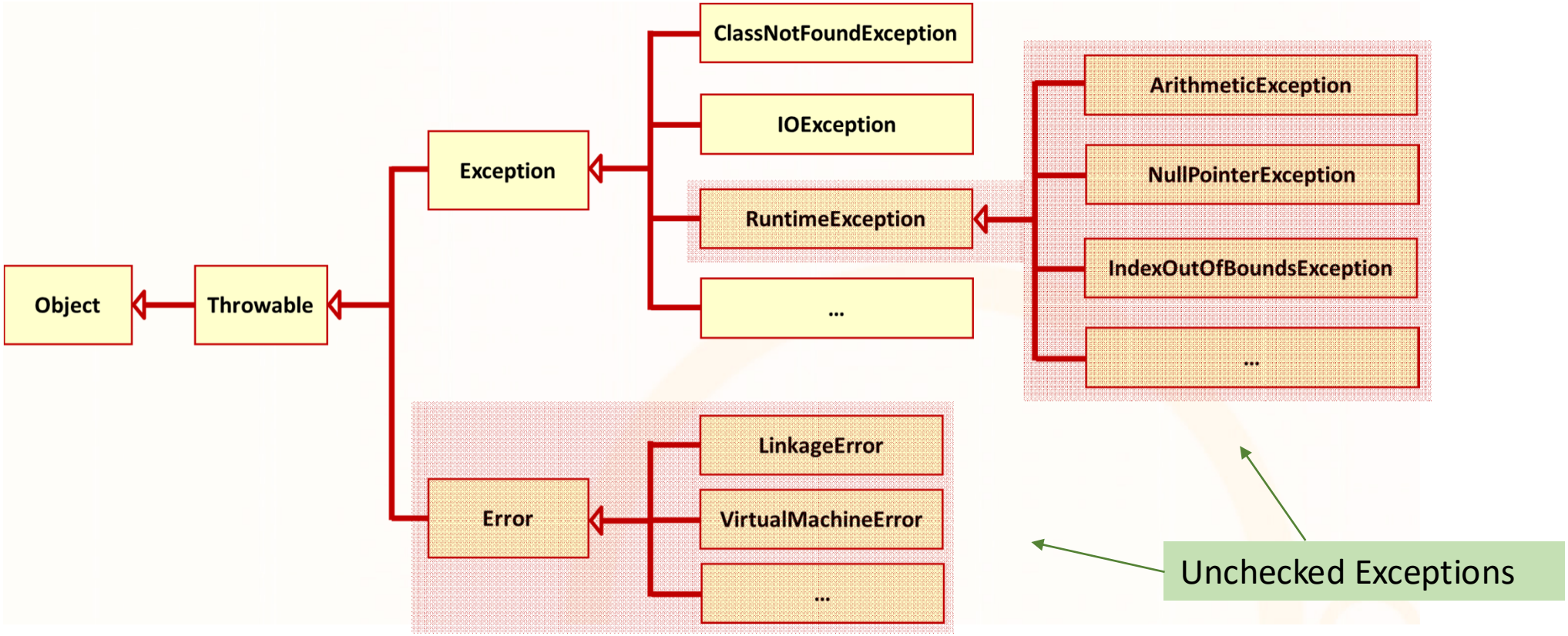
- Υπάρχει μια ολόκληρη κατηγορία από εξαιρέσεις οι οποίες εγείρονται αυτόματα από το σύστημα εκτέλεσης και για τις οποίες ο προγραμματιστής δεν υποχρεούται να τις λάβει υπόψη του στον προσδιορισμό των εξαιρέσεων των μεθόδων του.
- Οι εξαιρέσεις αυτού του τύπου αντιστοιχούν συνήθως είτε σε καταστάσεις τις οποίες δεν μπορεί να ελέγξει ο προγραμματιστής (π.χ. πέρασμα λάθος τιμής παραμέτρου σε μια μέθοδο του) είτε σε λογικά σφάλματα του προγράμματος.
- Εάν μια εξαίρεση τύπου RuntimeException δεν ανιχνευθεί από κάποιο διαχειριστή εξαίρεσης, θα “φθάσει” μέχρι το πλαίσιο συμφραζομένων της main, και θα προκαλέσει έξοδο του προγράμματος με προηγούμενη κλήση της PrintStackTrace, για διευκόλυνση της αποσφαλμάτωσης.

Checked vs. Unchecked Exceptions

- **Unchecked Exceptions**: Οι εξαιρέσεις των κλάσεων **RuntimeException** και **Error.**
  - Συνήθως δεν μπορούν να προβλεφθούν και πολύ απίθανο να διαχειριστούν.
  - Δεν τις κτυπάει ο compiler, και δεν είναι αναγκαστικό να τις διαχειριστούμε.
    - Δεν σημαίνει όμως ότι δεν είναι σημαντικά αυτά τα λάθη.

- **Checked Exceptions**: όλα τα άλλα exceptions. Μπορούν να προβλεφθούν και να διαχειριστούν.

Checked vs. Unchecked Exceptions



Ένα απλό παράδειγμα

- Ένα πρόγραμμα σχολής χορού ταιριάζει χορευτές με χορεύτριες
  - Αν οι άνδρες είναι περισσότεροι από τις γυναίκες τότε ο καθένας θα χορέψει με πάνω από μία γυναίκα
  - Αν οι γυναίκες είναι παραπάνω από τους άνδρες τότε η κάθε μία θα χορέψει με παραπάνω από έναν άνδρα
  - Αν είναι μισοί μισοί, τότε ταιριάζονται ένας προς ένα
- Τι γίνεται **αν δεν υπάρχουν** άνδρες, ή γυναίκες, ή καθόλου μαθητές?
  - Αυτό είναι μια ειδική περίπτωση για την οποία δημιουργούμε μια εξαίρεση

Υλοποίηση χωρίς εξαιρέσεις

```
import java.util.Scanner;

public class DanceLesson
{
    public static void main(String[] args)
    {
        Scanner keyboard = new Scanner(System.in);
        System.out.println("Enter number of male and female dancers:");
        int men = keyboard.nextInt();
        int women = keyboard.nextInt();

        if (men == 0 && women == 0){
            System.out.println("Lesson is canceled. No students");
            System.exit(0);
        }else if (men == 0){
            System.out.println("Lesson is canceled. No men");
            System.exit(0);
        }else if (women == 0){
            System.out.println("Lesson is canceled. No women");
            System.exit(0);
        }

        if (women >= men)
            System.out.println("Each man must dance with " + women/(double)men + " women");
        else
            System.out.println("Each woman must dance with " + men/(double)women + " men");

        System.out.println("Begin the lesson.");
    }
}
```

Υλοποίηση με εξαιρέσεις

- Όταν υπάρχει πρόβλημα στην εκτέλεση του προγράμματος (π.χ., μηδενικός αριθμός από άνδρες ή γυναίκες) το πρόγραμμα μας θα πετάει (δημιουργεί) μια εξαίρεση (**throws an exception**) και σταματάει την ομαλή ροή του προγράμματος

- Σε κάποιο άλλο σημείο του προγράμματος πιάνουμε (χειριζόμαστε) την εξαίρεση (**catch the exception**) και έχουμε κώδικα που την χειρίζεται

- Τι είναι μια εξαίρεση?
  - Μια εξαίρεση είναι ένα **αντικείμενο** της κλάσης **Exception** ή κάποιας **παράγωγης κλάσης** της Exception που κρατάει πληροφορία για το τι προκάλεσε το πρόβλημα

Μηχανισμός try-throw-catch

- Ο κώδικας που μπορεί να δημιουργήσει εξαίρεση μπαίνει σε ένα **try-block**
- Αν η εξέλιξη του κώδικα είναι προβληματική εκτελείται η εντολή **throw** η οποία «πετάει» την εξαίρεση
- Το πέταγμα της εξαίρεσης μπορεί να γίνεται και από κάποια μέθοδο που καλείται μέσα στο try block
- Αν υπάρξει εξαίρεση η ροή του κώδικα μεταφέρεται στο **catch-block** το οποίο χειρίζεται τις εξαιρέσεις

```
try
{
   <Κώδικας πριν>

   <Κώδικας ο οποίος μπορεί να κάνει
        throw exception>

   <Κώδικας μετά>
}

catch (Exception e)
{
   <Κώδικας που χειρίζεται την εξαίρεση>
   <Χρησιμοποιεί το αντικείμενο e>
}
```

Το try block

- Σύνταξη

```
try
{
        <Κώδικας που μπορεί να προκαλέσει εξαίρεση>
}
```

- Το **try block** είναι ένα **block** όπως όλα τα άλλα στην Java
  - Ότι μεταβλητή ορίζεται μέσα στο block είναι τοπική, κλπ…

Η εντολή throw

- Σύνταξη

```
throw <Αντικείμενο της κλάσης Exception (ή παράγωγης)>
```

- Η εντολή **throw** λειτουργεί ως τελεστής, και ακολουθείται από ένα αντικείμενο τύπου **Exception**, ή παράγωγης κλάσης της Exception
  - Αυτή είναι η εξαίρεση που πετάει ο κώδικας
- Όταν πεταχτεί η εξαίρεση (π.χ., όταν κληθεί η throw) βγαίνουμε αυτόματα εκτός του try block και ο έλεγχος του προγράμματος μεταφέρεται στο αντίστοιχο **catch block**
  - Λειτουργεί όπως την break σε switch block

Η κλάση Exception

- Η κλάση **Exception** κρατάει πληροφορίες για την εξαίρεση που δημιουργήθηκε
  - Έχει ένα πεδίο **message** το οποίο κρατάει ένα μήνυμα για το πρόβλημα και το οποίο μπορούμε να διαβάσουμε με την μέθοδο **getMessage()**
  - Π.χ., όταν καλούμε τον constructor

    ```
    new Exception("No students. No Lesson");
    ```

    - Στο private πεδίο **message** της κλάσης **Exception** αποθηκεύεται το μήνυμα που δίνουμε ως όρισμα
- Μπορούμε να δημιουργήσουμε παράγωγες κλάσεις της Exception και να δημιουργήσουμε **επιπλέον πεδία** για ν κρατάμε περισσότερες πληροφορίες για κάποια εξαίρεση

Το catch block

- Σύνταξη

```
catch(Exception e)
{
        <Κώδικας που χειρίζεται την εξαίρεση>
}
```

- Η παράμετρος **Exception e** δηλώνει τον τύπο της εξαίρεσης που χειρίζεται το block και τη μεταβλητή e της εξαίρεσης (αντικείμενο)
- Χρησιμοποιώντας τη μεταβλητή μπορούμε να έχουμε πρόσβαση στα πεδία της εξαίρεσης
  - Παράδειγμα

```
catch(Exception e)
{
   String message = e.getMessage();
   System.out.println(message);
   System.exit(0);
}
```

Επιστρέφει το String του message

Try-throw-catch

- Σύνταξη

```
try
{
        <Κώδικας πριν>
        <Κώδικας ο οποίος μπορεί να κάνει throw exception>
        <Κώδικας μετά>
}
catch (Exception e)
{
     <Κώδικας που χειρίζεται την εξαίρεση>
}
```

- Μπαίνοντας στο try block, εκτελείται ο **κώδικας πριν**
- Αν υπάρχει εξαίρεση η ροή μεταφέρεται στο **catch block**
- Αν **δεν υπάρχει** εξαίρεση εκτελείται ο κώδικας μετά. Ο κώδικας του catch block **δεν εκτελείται ποτέ**.

Υλοποίηση με εξαιρέσεις

```
import java.util.Scanner;

public class DanceLesson2
{
    public static void main(String[] args)
    {
        Scanner keyboard = new Scanner(System.in);
        System.out.println("Enter number of male and female dancers:");
        int men = keyboard.nextInt();
        int women = keyboard.nextInt();

        try{
            if (men == 0 && women == 0)
                throw new Exception("Lesson is canceled. No students.");
            else if (men == 0)
                throw new Exception("Lesson is canceled. No men.");
            else if (women == 0)
                throw new Exception("Lesson is canceled. No women.");

            if (women >= men)
                System.out.println("Each man dances with " + women/(double)men + " women");
            else
                System.out.println("Each woman dances with " + men/(double)women + " men");
        }
        catch(Exception e){
            String message = e.getMessage( );
            System.out.println(message);
            System.exit(0);
        }
        System.out.println("Begin the lesson.");
    }
}
```

Σημείωση: Το παράδειγμα είναι ενδεικτικό. Στην πράξη ποτέ δεν θα χρησιμοποιούσατε εξαιρέσεις με αυτόν τον τρόπο και για ένα τόσο απλό πρόβλημα

Εξειδικευμένες εξαιρέσεις

- Η κλάση Exception είναι η πιο γενική κλάση εξαίρεσης
  - Υπάρχουν και πιο εξειδικευμένες κλάσεις εξαιρέσεων που κληρονομούν από την Exception σε διάφορα πακέτα της Java. Π.χ.
    - FileNotFoundException
    - IOException
- Μπορούμε επίσης να ορίσουμε και δικές μας κλάσεις εξαιρέσεων ανάλογα με τις ανάγκες μας
- Αυτό είναι χρήσιμο ώστε να έχουμε και εξειδικευμένα catch blocks όπως θα δούμε αργότερα

Παράδειγμα

- Θέλουμε να ορίσουμε μια εξαίρεση για την περίπτωση που προσπαθούμε να διαιρέσουμε με το μηδέν
  - Η κλάση DivisionByZeroException

- Η κλάση μας θα κληρονομεί από την Exception οπότε θα έχει την μέθοδο getMessage() για να επιστρέφει το μήνυμα
  - Συνήθως το μόνο που χρειάζεται είναι να ορίσουμε τον constructor

Παράδειγμα

```
public class DivisionByZeroException extends Exception
{
    public DivisionByZeroException( )
    {
        super("Division by Zero!");
    }

    public DivisionByZeroException(String message)
    {
        super(message);
    }
}
```

Η κλάση κληρονομεί και την μέθοδο getMessage()

```
import java.util.Scanner;

public class DivisionDemoFirstVersion
{

    public static void main(String[] args)
    {
        try
        {
            Scanner keyboard = new Scanner(System.in);

            System.out.println("Enter numerator:");
            int numerator = keyboard.nextInt();
            System.out.println("Enter denominator:");
            int denominator = keyboard.nextInt();

            if (denominator == 0)
                throw new DivisionByZeroException( );

            double quotient = numerator/(double)denominator;
            System.out.println(numerator + "/" + denominator + " = " + quotient);
        }
        catch(DivisionByZeroException e)
        {
            System.out.println(e.getMessage( ));
            system.Exit(0);
        }

        System.out.println("End of program.");
    }
}
```

```
import java.util.Scanner;

public class DivisionDemoFirstVersion
{

    public static void main(String[] args)
    {
        try
        {
            Scanner keyboard = new Scanner(System.in);

            System.out.println("Enter numerator:");
            int numerator = keyboard.nextInt();
            System.out.println("Enter denominator:");
            int denominator = keyboard.nextInt();

            if (denominator == 0)
                throw new DivisionByZeroException( );

            double quotient = numerator/(double)denominator;
            System.out.println(numerator + "/" + denominator + " = " + quotient);
        }
        catch(DivisionByZeroException e)
        {
            System.out.println(e.getMessage( ));
            secondChance( );
        }

        System.out.println("End of program.");
    }
}
```

Μπορούμε μέσα στο catch block να καλούμε μία άλλη μέθοδο

```
public static void secondChance( )
    {
        Scanner keyboard = new Scanner(System.in);

        System.out.println("Try again:");
        System.out.println("Enter numerator:");
        int numerator = keyboard.nextInt();
        System.out.println("Enter denominator:");
        System.out.println("Be sure the denominator is not zero.");
        int denominator = keyboard.nextInt();

        if (denominator == 0)
        {
            System.out.println("I cannot do division by zero.");
            System.out.println("Aborting program.");
            System.exit(0);
        }

        double quotient = ((double)numerator)/denominator;
        System.out.println(numerator + "/" + denominator  + " = " + quotient);
    }
}
```

Ορίζοντας Exceptions

- Ορίζουμε μια νέα εξαίρεση μόνο **αν υπάρχει ανάγκη**, αλλιώς μπορούμε να χρησιμοποιήσουμε την κλάση Exception
- Στη νέα κλάση ορίζουμε πάντα ένα **constructor χωρίς ορίσματα** και έναν που παίρνει το String του μηνύματος
- Διατηρούμε την μέθοδο **getMessage()** ως έχει
  - Συνήθως δεν θα χρειαστούμε κάποια άλλη μέθοδο

Εξαιρέσεις με επιπλέον πληροφορία

- Μια εξαίρεση συνήθως έχει ένα μήνυμα σε μορφή String
  - Μπορεί να έχει και επιπλέον πληροφορία η οποία αποθηκεύεται σε πεδία της κλάσης της εξαίρεσης

- Παράδειγμα: Ζητάμε το έτος γέννησης και θέλουμε να πετάμε μια εξαίρεση αν είναι μεγαλύτερο από 2025
  - Θα ορίσουμε το BadNumberException
  - Η εξαίρεση θα μεταφέρει πληροφορία για τον αριθμό που δόθηκε

```
public class BadNumberException extends Exception
{
    private int badNumber;

    public BadNumberException(int number)
    {
        super("BadNumberException");
        badNumber = number;
    }

    public BadNumberException( )
    {
        super("BadNumberException");
    }

    public BadNumberException(String message)
    {
        super(message);
    }

    public int getBadNumber( )
    {
        return badNumber;
    }
}
```

```
import java.util.Scanner;

public class BadNumberExceptionDemo
{

    public static void main(String[] args)
    {
        try
        {
            Scanner keyboard = new Scanner(System.in);

            System.out.println("Enter year of birth:");
            int inputNumber = keyboard.nextInt();

            if (inputNumber > 2025)
                throw new BadNumberException(inputNumber);

            System.out.println("Thank you for entering " + inputNumber);
        }
        catch(BadNumberException e)
        {
            System.out.println(e.getBadNumber( ) + " is not valid.");
        }

        System.out.println("End of program.");
    }
}
```

Μας επιστρέφει τον αριθμό που προκάλεσε την εξαίρεση

Πολλαπλά catch blocks

- Εφόσον έχουμε πολλαπλά είδη εξαιρέσεων είναι δυνατόν ένα try block να πετάει παραπάνω από ένα τύπο εξαίρεσης
- Στην περίπτωση αυτή χρειαζόμαστε και διαφορετικά catch blocks
- Εάν προκληθεί εξαίρεση, ο μηχανισμός διαχείρισής της ψάχνει για τον πρώτο διαχειριστή (catch), η παράμετρος του οποίου συμπίπτει με τον τύπο της εξαίρεσης

```
try {
    //Κώδικας που μπορεί να
    //δημιουργήσει exceptions
}
catch (ExceptionType1 et1 ) {
    //Διαχείριση/Αντιμετώπιση
    //exception τύπου Type1
}
catch (ExceptionType2 et2 ) {
    //Διαχείριση/Αντιμετώπιση
    //exception τύπου Type2
}
catch (ExceptionType3 et3 ) {
    //Διαχείριση/Αντιμετώπιση
    //exception τύπου Type3
}
catch (Exception ex ) {
    //Διαχείριση/Αντιμετώπιση
    //γενικού τύπου exception
}
```

```
public class NegativeNumberException extends Exception
{
    public NegativeNumberException( )
    {
        super("Negative Number Exception!");
    }

    public NegativeNumberException(String message)
    {
        super(message);
    }
}
```

```
try
{
    System.out.println("How many pencils do you have?");
    int pencils = keyboard.nextInt();

    if (pencils < 0)
        throw new NegativeNumberException("pencils");

    System.out.println("How many erasers do you have?");
    int erasers = keyboard.nextInt();
    double pencilsPerEraser;

    if (erasers < 0)
        throw new NegativeNumberException("erasers");
    else if (erasers != 0)
        pencilsPerEraser = pencils/(double)erasers;
    else
        throw new DivisionByZeroException( );

    System.out.println("Each eraser must last through " + pencilsPerEraser + " pencils");
    }

    catch(NegativeNumberException e)
    {
        System.out.println("Cannot have a negative number of " + e.getMessage( ));
    }
    catch(DivisionByZeroException e)
    {
        System.out.println("No erasers. Do not make any mistakes");
    }
```

Προσοχή

- Όταν γίνει trigger μια εξαίρεση και βγούμε από ένα try block, τα **catch blocks** εξετάζονται με την σειρά που εμφανίζονται στον κώδικα
- Θα εκτελεστεί το **πρώτο catch block** με όρισμα που ταιριάζει στο **exception** που έγινε triggered
- Για να είμαστε σίγουροι ότι θα εκτελεστεί το σωστό **catch block** θα πρέπει να έχουμε τις πιο **συγκεκριμένες εξαιρέσεις πρώτες** και τις **πιο γενικές μετά**
  - Αν είναι ανάποδα, οι πιο συγκεκριμένες εξαιρέσεις δεν θα εκτελεστούν **ποτέ**

```
import java.util.Scanner;

public class BadNumberExceptionDemo2
{

    public static void main(String[] args)
    {
        try
        {
            Scanner keyboard = new Scanner(System.in);

            System.out.println("Enter year of birth:");
            int inputNumber = keyboard.nextInt();
            if (inputNumber <=1973)
               throw new Exception("You are too old");
            if (inputNumber > 2025)
               throw new BadNumberException(inputNumber);

            System.out.println("Thank you for entering " + inputNumber);
        }
        catch(Exception e){
            System.out.println(e.getMessage());
        }
        catch(BadNumberException e){
            System.out.println(e.getBadNumber( ) +" not valid");
        }

        System.out.println("End of program.");
    }
}
```

Η εντολή throw δεν μας «στέλνει» στο σωστό catch block

Όταν πετάξει εξαίρεση, το πρόγραμμα παίρνει τα catch blocks με την σειρά και μπαίνει στο πρώτο που ταιριάζει με την εξαίρεση που πέταξε

Το BadNumberException «**είναι και**» Exception και άρα θα μπει σε αυτό το block

Ο compiler θα μας χτυπήσει λάθος γιατί δεν γίνεται ποτέ να μπούμε στο δεύτερο catch block

```
import java.util.Scanner;

public class BadNumberExceptionDemo3
{

    public static void main(String[] args)
    {
        try
        {
            Scanner keyboard = new Scanner(System.in);

            System.out.println("Enter year of birth:");
            int inputNumber = keyboard.nextInt();
            if (inputNumber <=1973)
               throw new Exception("You are too old");
            if (inputNumber > 2023)
               throw new BadNumberException(inputNumber);

            System.out.println("Thank you for entering " + inputNumber);
        }
        catch(BadNumberException e){
            System.out.println(e.getBadNumber( ) +" not valid");
        }
        catch(Exception e){
            System.out.println(e.getMessage());
        }

        System.out.println("End of program.");
    }
}
```

Η σωστή υλοποίηση

Πρώτα η πιο ειδική εξαίρεση και μετά η πιο γενική εξαίρεση

Η εντολή finally

- Το **finally block** εκτελείται πάντα!
- Εξασφαλίζει ότι κάποιες δηλώσει θα εκτελεστούν άσχετα αν συμβεί κάποιο απρόσμενο exception
- Μπορεί να χρησιμοποιηθεί και χωρίς δηλώσεις
- Επιτρέπει την εκτέλεση κώδικα ακόμα και στις περιπτώσεις που εκτελείτε κατά λάθος κάποιο return, continue ή break
- Πολύ καλή πρακτική

```
try {
    //Κώδικας που μπορεί να
    //δημιουργήσει exceptions
}
catch (ExceptionType1 et1 ) {
    //Διαχείριση/Αντιμετώπιση
    //exception τύπου Type1
}
…
catch (Exception ex ) {
    //Διαχείριση/Αντιμετώπιση
    //γενικού τύπου exception
}

finally {
    //Εκτελείτε πάντα
}
```

Μέθοδοι που πετάνε εξαιρέσεις

- Μέχρι τώρα είδαμε παραδείγματα όπου οι εξαιρέσεις πετιόνται και πιάνονται στον ίδιο κώδικα
  - Αυτό δεν είναι και τόσο ρεαλιστικό σενάριο

- Το πιο σύνηθες είναι ότι την εξαίρεση την πετάμε **σε μια μέθοδο** και την **πιάνουμε σε μία άλλη** μέθοδο

Μέθοδος που πετάει εξαίρεση

- Σύνταξη

```
ReturnType methodName(argument list) throws Exception
{
        <Κώδικας πριν>
        <Κώδικας ο οποίος κάνει throw Exception>
        <Κώδικας μετά>
}
```

- Αν η μέθοδος πετάξει μια εξαίρεση τότε σταματάει η εκτέλεση του κώδικα στο σημείο που πετάει την εξαίρεση
  - Με τον ίδιο τρόπο όπως η εντολή return

Μέθοδος που πετάει εξαίρεση

- Μία μέθοδος μπορεί να πετάει πολλές εξαιρέσεις

- Σύνταξη:

```
ReturnType methodName(argument list)
        throws Exception1, Exception2
{
      <Κώδικας πριν>
      <Κώδικας ο οποίος κάνει throw Exception1>
      <Κώδικας μετά>
      <Κώδικας ο οποίος κάνει throw Exception2>
      <Κώδικας μετά>
}
```

```
import java.util.Scanner;

public class DivisionDemoSecondVersion
{
    public static void main(String[] args)
    {
        Scanner keyboard = new Scanner(System.in);

        try
        {
            System.out.println("Enter numerator and denominator :");
            int numerator = keyboard.nextInt(); int denominator = keyboard.nextInt();

            double quotient = safeDivide(numerator, denominator);
            System.out.println(numerator + "/" + denominator + " = " + quotient);
        }
        catch(DivisionByZeroException e)
        {
            System.out.println(e.getMessage( ));
            secondChance();
        }

        System.out.println("End of program.");
    }


    public static double safeDivide(int top, int bottom) throws DivisionByZeroException
    {
        if (bottom == 0)
            throw new DivisionByZeroException( );

        return top/(double)bottom;
    }
}
```

Εφόσον έχουμε μία μέθοδο που πετάει εξαίρεση, **πρέπει** να τη βάλουμε μέσα σε **try-catch block**

Η εξαίρεση δημιουργείται στην **safeDivide** αλλά την πιάνουμε και την χειριζόμαστε στην main

Ο προσδιορισμός των εξαιρέσεων

- Η Java ενθαρρύνει την ενημέρωση των μεθόδων με τις πιθανές εξαιρέσεις που μπορούν να εγείρουν και δεν τις διαχειρίζονται
- Για τον σκοπό αυτό, το συντακτικό της γλώσσας παρέχει την δήλωση throws με την οποία δηλώνεται ποιες εξαιρέσεις μπορεί να εγερθούν στο σώμα κάποιας μεθόδου
- Η χρήση της throws είναι υποχρεωτική και επιβάλλεται από τον μεταγλωττιστή, ακολουθώντας τη δήλωση των παραμέτρων της μεθόδου:

```
void f() throws TooBig, TooSmall, DivZero { …       }
```

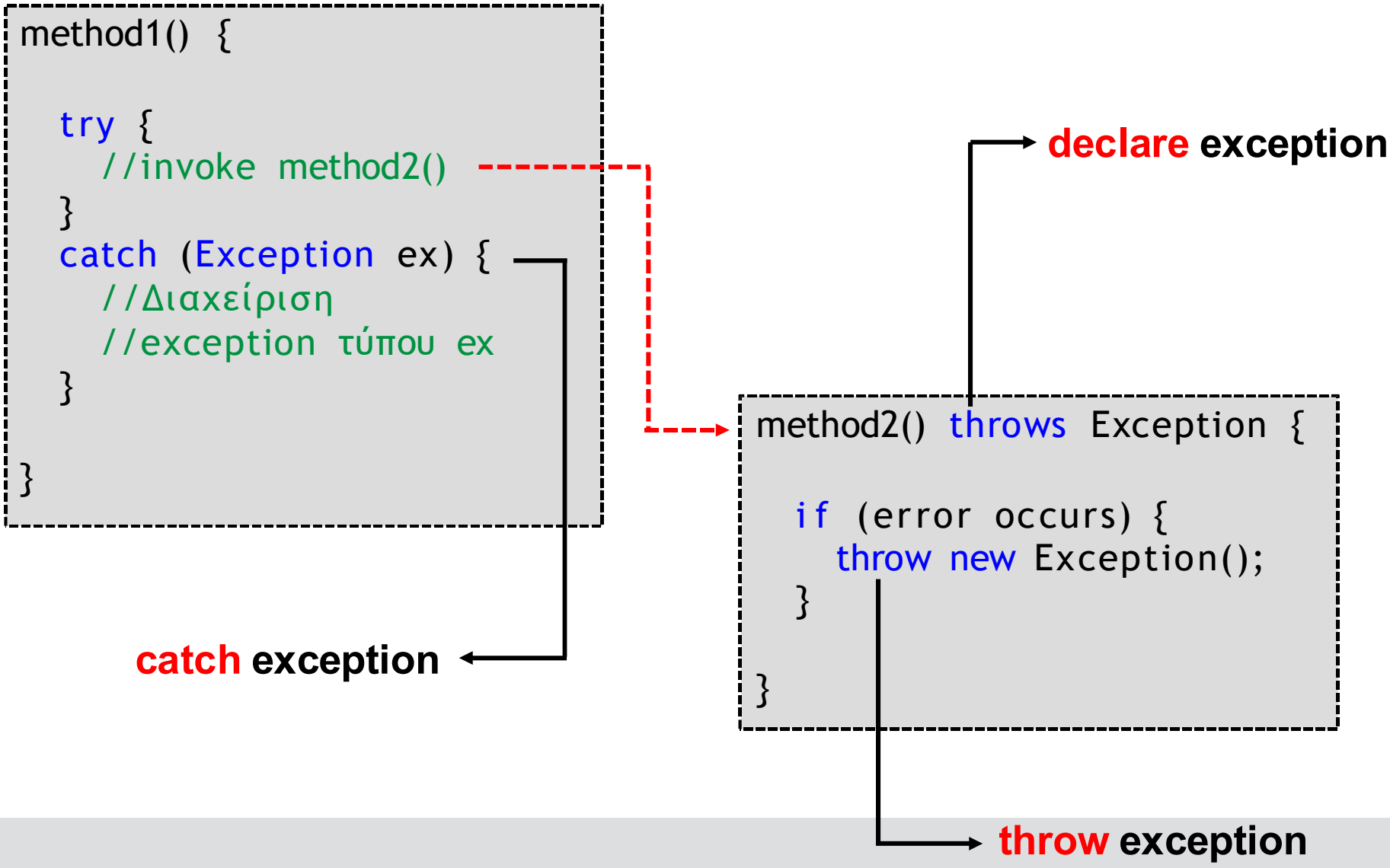
- Υπάρχει η δυνατότητα μια μέθοδος να δηλώσει ότι εγείρει κάποια εξαίρεση ενώ **δεν** περιλαμβάνει στο σώμα της την αντίστοιχη εντολή throw ;
  - Γιατί παρέχεται η δυνατότητα αυτή;

Catch or Declare

- Μια μέθοδος η οποία **καλεί** μια άλλη μέθοδο που **πετάει εξαίρεση** έχει δύο επιλογές
  - Catch: Να **πιάσει** και να **χειριστεί** την εξαίρεση
  - Declare: Να κάνει κι αυτή **throw** την εξαίρεση
    - Αυτό είναι μια μορφή **μετάθεσης ευθυνών**, αφήνουμε την παραπάνω μέθοδο να χειριστεί την εξαίρεση
- Αν δεν κάνουμε ένα από τα δύο, ο compiler θα παραπονεθεί

- Εξαίρεση: Runtime exceptions
  - Κάποιες εξαιρέσεις μπορούμε απλά να τις **αφήσουμε**. Αν συμβούν το πρόγραμμα μας θα τερματίσει με λάθος
  - Π.χ., **NullPointerException**

Declaring, Throwing, Catching Exceptions



```
import java.util.Scanner;

public class DivisionDemoSecondVersion
{
    public static void main(String[] args)
    {
          Scanner keyboard = new Scanner(System.in);

          try
          {
              System.out.println("Enter numerator, denominator :");
              int numerator = keyboard.nextInt(); int denominator = keyboard.nextInt();

              int percentage = safePercentage(numerator, denominator);
              System.out.println("percentage  = " + percentage +"%");
          }
          catch(DivisionByZeroException e)
          {
               System.out.println(e.getMessage( ));
               secondChance();
          }
    }

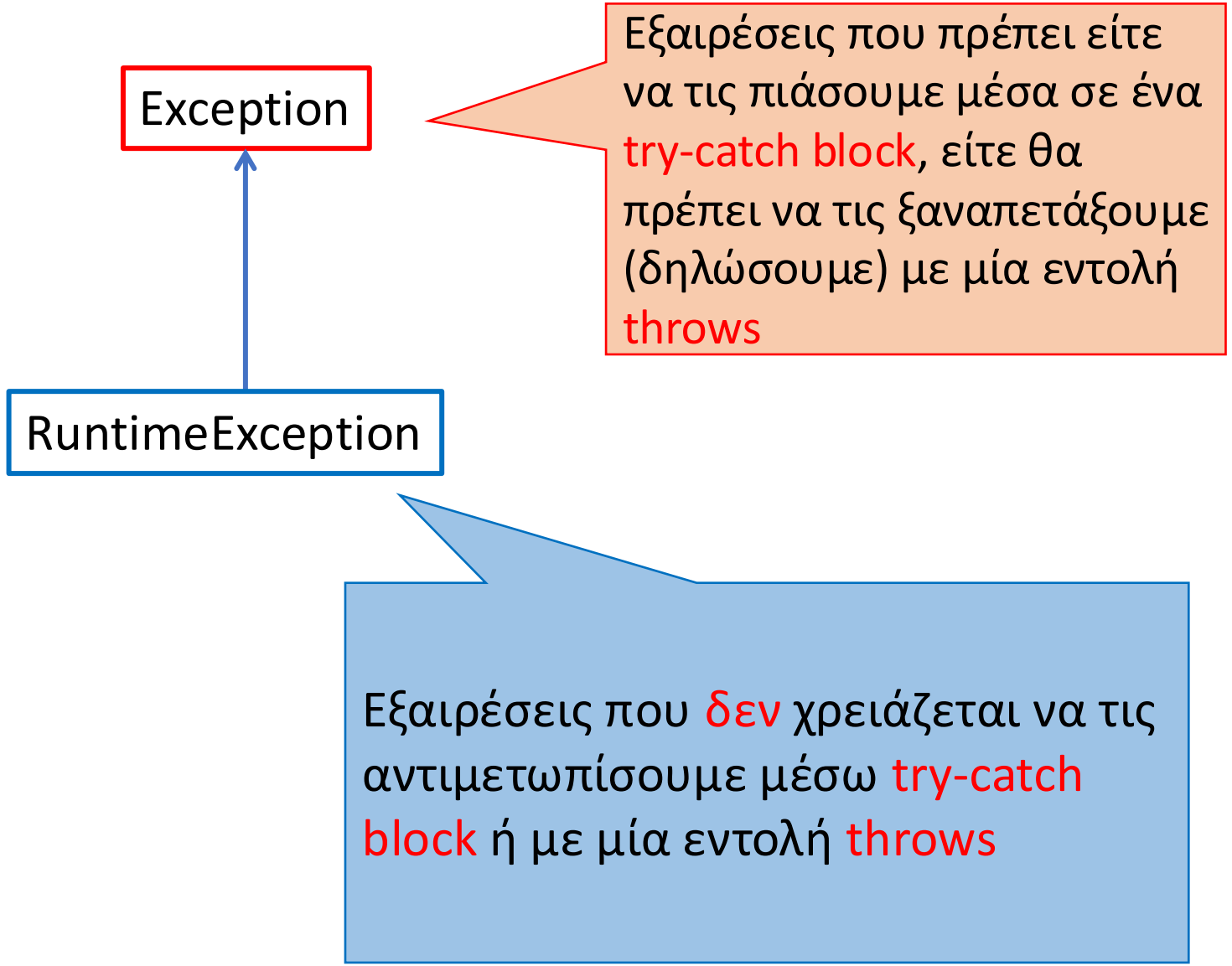
    public static int safePercentage(int top, int bottom) throws DivisionByZeroException
    {
         double ratio = safeDivide(top,bottom);
         return (int)(ratio*100);
    }

    public static double safeDivide(int top, int bottom) throws DivisionByZeroException
    {
         if (bottom == 0)
              throw new DivisionByZeroException( );
         return top/(double)bottom;
    }
}
```

Εφόσον η main δεν πετάει εξαίρεση, θα πρέπει να βάλουμε την κλήση της **safePercentage** μέσα σε try-catch block

Η safePercentage δεν χρειάζεται try-catch block γιατί πετάει κι αυτή την εξαίρεση της safeDivide (declare). Αλλιώς θα είχαμε compile error.

Τύποι Εξαιρέσεων



```
import java.util.Scanner;
import java.util.InputMismatchException;

public class InputMismatchExceptionDemo
{
    public static void main(String[] args)
    {
        Scanner keyboard = new Scanner(System.in);
        int number = 0; //to keep compiler happy
        boolean done = false;

        while (!done)
        {
            try
            {
                System.out.println("Enter a whole number:");
                number = keyboard.nextInt();
                done = true;
            }
            catch(InputMismatchException e)
            {
                keyboard.nextLine();
                System.out.println("Not a correctly written whole number.");
                System.out.println("Try again.");
            }
        }

        System.out.println("You entered " + number);
    }
}
```

Αν και δεν είναι απαραίτητο μπορούμε να πιάσουμε ένα **RuntimeException**.

Στο παράδειγμα αυτό χρησιμοποιούμε το **InputMismatchException** για να δημιουργήσουμε ένα βρόχο μέχρι να δοθεί το σωστό input

Η εξαίρεση δημιουργείται από την μέθοδο nextInt()

Το **InputMismatchException** είναι υπάρχουσα **RuntimeException** της Java

```
import java.util.Scanner;
import java.util.InputMismatchException;

public class InputMismatchExceptionDemo2
{
    public static void main(String[] args)
    {
        Scanner keyboard = new Scanner(System.in);
        int number = 0; //to keep compiler happy


        while (true)
        {
            try
            {
               System.out.println("Enter a whole number:");
               number = keyboard.nextInt();
               break;
            }
            catch(InputMismatchException e)
            {
                keyboard.nextLine();
                System.out.println("Not a correctly written whole number.");
                System.out.println("Try again.");
            }
        }

        System.out.println("You entered " + number);
    }
}
```

Άλλος τρόπος να κάνουμε τον ίδιο κώδικα χρησιμοποιώντας την break

Η εντολή try-with-resources

- Παρόμοιο με τη δήλωση try αλλά επιτρέπει και τη αρχικοποίηση κάποιων πόρων (resources)
- Ένας πόρος (resource) είναι ένα αντικείμενο το οποίο υπάρχει μόνο κατά την εκτέλεση του code block
- Μόνο τα αντικείμενα που υλοποιούν το interface **java.lang.AutoCloseable** μπορούν να χρησιμοποιηθούν σαν resource (περιλαμβάνει όλα τα αντικείμενα που υλοποιούν **java.io.Closeable**)

```
try ( BufferedReader br =
        new BufferedReader(
         new FileReader(path))) {
    //Κώδικας που μπορεί να
    //δημιουργήσει exceptions
}
…
```

Η εντολή try-with-resources

```
Scanner scanner = null;
try {
    scanner = new Scanner(new File("test.txt"));
    while (scanner.hasNext()) {
        System.out.println(scanner.nextLine());
        }
    } catch (FileNotFoundException e) {
        e.printStackTrace();
    } finally {
        if (scanner != null) {
            scanner.close();
        }
    }
```

```
try (Scanner scanner = new Scanner(new File("test.txt")))
{
    while (scanner.hasNext()) {
        System.out.println(scanner.nextLine());
    }
} catch (FileNotFoundException fnfe) {
    fnfe.printStackTrace();
}
```

try-with-resources with multiple resources

```
try (Scanner scanner = new Scanner(new File("testRead.txt"));
    PrintWriter writer = new PrintWriter(new File("testWrite.txt"))) {
    while (scanner.hasNext()) {
        writer.print(scanner.nextLine());
    }
} catch (FileNotFoundException fnfe) {
    fnfe.printStackTrace();
}
```

Περιορισμοί εξαιρέσεων

- Όταν κάνουμε **override** μια μέθοδο (κληρονομικότητα), μπορούμε να εγείρουμε μόνο εξαιρέσεις οι οποίες καθορίζονται στην **parent κλάση** της μεθόδου που γίνεται override
- Ο περιορισμός αυτός υπάρχει ούτως ώστε κώδικες που δουλεύουν με μια κλάση, να δουλεύουν σωστά και με κάθε αντικείμενο που κληρονομεί από αυτή την κλάση
- Οι περιορισμοί κληρονομικότητας των εξαιρέσεων δεν ισχύουν για τους **constructors**. Ένας **constructor** μπορεί να εγείρει ότι εξαιρέσεις θέλει
  - Ωστόσο, επειδή η κλήση του εμπεριέχει κλήση στον constructor της **parent κλάσης**, ο constructor πρέπει να δηλώνει όλες τις εξαιρέσεις του constructor της **parent** στον προσδιορισμό των δικών του εξαιρέσεων
  - Σημειώστε ότι ένας constructor δεν μπορεί να ανιχνεύει τις εξαιρέσεις που εγείρει ο constructor της **parent κλάσης**

Παρατηρήσεις

- Δεν σημαίνει ότι μία δήλωση exception που υπάρχει σε μια μέθοδο της **parent κλάσης** θα πρέπει να υπάρχει και στην μέθοδο που ορίζεται στην υποκλάση
- Αυτό είναι διαφορετικό από τους κανόνες κληρονομικότητας που καθορίζουν ότι μία μέθοδος στην **parent κλάση** πρέπει να υπάρχει και στην υποκλάση
- Με απλά λόγια, οι προδιαγραφές μίας εξαίρεσης για κάποια συγκεκριμένη μέθοδο μπορούν να "**στενέψουν**" κατά τη διάρκεια της κληρονομικότητας αλλά όχι να διευρυνθούν

Χρήση εξαιρέσεων σε βρόχους

- Μπορούμε να χρησιμοποιούμε τις εξαιρέσεις για να δημιουργήσουμε συνθήκες σε βρόχους όπως είδαμε παραπάνω ώστε να εξασφαλίσουμε την λειτουργία του προγράμματος όπως την θέλουμε

Χρήση Εξαιρέσεων

- Τις εξαιρέσεις θα τις δείτε περισσότερο όταν θα πρέπει να χρησιμοποιήσετε κάποια βιβλιοθήκη που έχει μεθόδους που πετάνε εξαιρέσεις

- Έχει νόημα να πετάμε μια εξαίρεση όταν έχουμε μία μέθοδο που δεν ξέρει πώς να χειριστεί ένα λάθος, και η απόφαση θα πρέπει να παρθεί σε κάποιο άλλο σημείο του κώδικα που έχουμε περισσότερες πληροφορίες
  - Για παράδειγμα δεν είναι δουλειά της **safeDivide** να ξαναζητήσει τους αριθμούς. Αφήνει την main να το κάνει

Προσοχή

- Η εύκολη και τεμπέλικη λύση για μια εξαίρεση είναι να την πιάσουμε και απλά να μην κάνουμε τίποτα, αλλά αυτό είναι κακή προγραμματιστική τακτική